TransitOps

Stack Rationale, Libraries, and Build Plan

TransitOps Team

July 2026 · v1.0

1. Stack Summary

Layer	Choice
Frontend	Next.js 14 (App Router) + TypeScript + Tailwind CSS
Charts	Chart.js via react-chartjs-2
Backend	Django 5 + Django REST Framework
Auth	djangorestframework-simplejwt (JWT)
Database	PostgreSQL + Django ORM
Background jobs	Celery + Redis (fallback: synchronous management command)
Data/reporting	Pandas (aggregations, CSV export)
Drag-and-drop (stretch)	dnd-kit
Dev/infra	Docker + docker-compose, Git

This keeps the stack you proposed (Django, Next.js, CSS, Pandas, Chart.js) and adds only what's necessary to make the Ecount-style enhancements (reminders, mileage anomaly detection) actually work, without over-engineering for an 8-hour build.

2. Frontend

2.1 Core

- **Next.js 14 (App Router, TypeScript)** — file-based routing maps cleanly onto the 9 wireframe screens; API routes not needed since DRF is the backend, but Next's server components help for the initial dashboard data fetch.
- **Tailwind CSS** — matches the clean, utility-driven look of the wireframe (badges, tables, filter bars) without hand-writing custom CSS under time pressure.
- **React Hook Form + Zod** — form state + schema validation for Trip creation, Vehicle registration, etc.; mirrors backend validation messages 1:1 (e.g., capacity exceeded).

2.2 Data & State

- **Axios** — API client with interceptor for JWT attach/refresh.
- **TanStack Query (React Query)** — server-state caching for lists (vehicles, drivers, trips) with automatic refetch after mutations (e.g., after dispatch, refetch vehicle list so status updates reflect immediately).

2.3 Visualization

- **Chart.js (react-chartjs-2)** — Monthly Revenue bar chart, Top Costliest Vehicles bar chart, Vehicle Status distribution, Mileage trend line with anomaly markers.

2.4 Stretch UI

- **dnd-kit** — 2D tyre position drag-and-drop diagram (lightweight vs react-dnd, better touch support, easier to wire up quickly).

3. Backend

3.1 Core

- **Django 5** — batteries-included ORM, admin panel (useful for quickly seeding/inspecting demo data during the hackathon), migrations.
- **Django REST Framework (DRF)** — ViewSets + Routers for standard CRUD (Vehicles, Drivers, Maintenance, Expenses), custom `@action` endpoints for state-transition actions (`/trips/{id}/dispatch/`, `/complete/`, `/cancel/`).
- **djangorestframework-simplejwt** — JWT auth; pairs with custom DRF permission classes for RBAC.
- **django-cors-headers** — allow the Next.js dev/prod origin.
- **django-filter** — query-param filtering for Dashboard filters (vehicle type/status/region) and Trip search.
- **drf-spectacular** — auto-generated OpenAPI/Swagger docs — useful for frontend/backend parallel development during the hackathon (frontend can build against the schema before every endpoint is finished).

3.2 Business Logic Layer

- Plain Python **service modules** (`services/trip_service.py`, `services/fuel_service.py`, `services/maintenance_service.py`) called from DRF views — keeps business-rule code (capacity checks, status transitions, mileage calc) testable and out of serializers/views.

3.3 Background Processing

- **Celery + Redis** — scheduled tasks (Celery beat): daily document/license-expiry scan, mileage rolling-average recompute after each fuel log, depot-return-overdue scan.
- **Fallback if time-constrained**: skip Celery entirely; run the same logic synchronously via a Django management command (`python manage.py scan_reminders`) invoked on dashboard load, cached 5 minutes with Django's cache framework (`locmem` backend — zero extra infra). Functionally the same for a live demo.

4. Database

- **PostgreSQL** — relational integrity for the many FK relationships (Trip → Vehicle/Driver, Tyre → TyrePositionHistory), and native support for `SELECT ... FOR UPDATE` row locking used in the service layer to prevent double-dispatch race conditions.
- **Django ORM** — migrations, no raw SQL needed for MVP scope; use `select_related`/`prefetch_related` on list endpoints (Vehicles, Trips) to avoid N+1 queries on the dashboard.

5. Data & Reporting

- **Pandas** — used server-side inside the `/reports/analytics/` and `/reports/export.csv` endpoints to aggregate FuelLog/Expense/MaintenanceRecord querysets into the KPI figures (fuel efficiency, utilization %, ROI, category-wise cost breakdown) and to generate the CSV export cleanly (`DataFrame.to_csv()`).
- CSV export is mandatory per the PS; PDF export (bonus) can be added later via `reportlab` if time remains, reusing the same Pandas-aggregated data.

6. Auth & RBAC Implementation

- Login issues JWT access + refresh tokens; role is embedded in the JWT payload (read-only claim) and re-verified server-side on every request against the `User.role` DB field (never trust the token claim alone for authorization — refetch role from DB on sensitive actions).
- Custom DRF permission classes, e.g. `IsFleetManager`, `IsDispatcherOrReadOnly`, composed per ViewSet to match the RBAC matrix in the Design Doc.
- Frontend: role stored in an auth context/store (Zustand or React Query cache); navigation items conditionally rendered per role, but this is a UX convenience only — actual enforcement is server-side.

7. Dev Tooling & Deployment

Purpose	Tool
Containerization	Docker + docker-compose (services: `web` Django, `frontend` Next.js, `db` Postgres, `redis`, `worker` Celery)
API docs/testing	drf-spectacular (Swagger UI) + Postman collection
Linting/formatting	Black + isort (Python), ESLint + Prettier (TS)
Version control	Git, feature-branch workflow given hackathon team split
Demo hosting	Railway/Render for backend + Postgres, Vercel for Next.js frontend — fastest path to a shareable demo URL

8. Suggested Folder Structure

```
transitops/
├── backend/
│   ├── config/                # Django settings, urls, celery.py
│   ├── apps/
│   │   ├── accounts/          # User, Role, RBAC
│   │   ├── vehicles/          # Vehicle, VehicleDocument, Tyre, TyrePositionHistory
│   │   ├── drivers/
│   │   ├── trips/
│   │   ├── maintenance/
│   │   ├── finance/           # FuelLog, Expense, ROI calc
│   │   ├── notifications/
│   │   └── reports/           # Pandas aggregation + CSV export
│   ├── services/              # cross-cutting business logic
│   └── manage.py
├── frontend/
│   ├── app/
│   │   ├── (auth)/login/
│   │   ├── dashboard/
│   │   ├── fleet/
│   │   ├── drivers/
│   │   ├── trips/
│   │   ├── maintenance/
│   │   ├── fuel-expenses/
│   │   ├── analytics/
│   │   └── settings/
│   ├── components/
│   ├── lib/                   # axios client, react-query hooks
│   └── styles/
└── docker-compose.yml
```

9. Library Reference Table

Library	Version (approx.)	Purpose
django	5.x	Core web framework
djangorestframework	3.15+	REST API layer
djangorestframework-simplejwt	5.x	JWT auth
django-cors-headers	4.x	CORS for Next.js origin
django-filter	24.x	Query filtering

drf-spectacular	0.27+	OpenAPI/Swagger docs
psycopg2-binary	2.9+	Postgres driver
celery	5.x	Background task queue (optional)
redis	5.x	Celery broker (optional)
pandas	2.x	Analytics aggregation, CSV export
next	14.x	Frontend framework
react / react-dom	18.x	UI runtime
typescript	5.x	Type safety
tailwindcss	3.x	Styling
axios	1.x	HTTP client
@tanstack/react-query	5.x	Server-state caching
react-hook-form + zod	latest	Forms + validation
chart.js + react-chartjs-2	latest	Charts
@dnd-kit/core	latest	Drag-and-drop (tyre diagram, stretch)

10. Why This Stack Over Alternatives

- **Django+DRF vs Node/Express**: Django's ORM + admin panel drastically cuts boilerplate for the 8+ entity data model here, and DRF's serializer validation maps directly onto the "business rule enforcement" requirement graded in the PS.
- **Next.js vs plain React (CRA/Vite)**: App Router's nested layouts fit the sidebar-navigation wireframe well (shared layout + role-based route groups), and it's a safer long-term choice if the project continues past the hackathon.
- **Celery vs cron-only**: Celery beat gives "true" scheduled scans; however, given time risk, it's explicitly optional — the fallback in §3.3 (Design Doc §9.3) ensures the reminder features still work in the demo even if Celery setup is skipped.
- **Pandas vs raw SQL aggregation**: faster to iterate on ROI/utilization formulas mid-hackathon without writing/debugging complex SQL under time pressure.

11. 8-Hour Build Timeline (Suggested Task Split)

Hour	Backend	Frontend
0–1	Project scaffold, models (Vehicle, Driver, Trip, User/Role), migrations	Project scaffold, layout/sidebar, auth pages
1–2	Auth (JWT), RBAC permission classes	Login flow, protected routes, role-based nav
2–3	Vehicle + Driver CRUD APIs	Vehicle Registry + Driver screens
3–4	Trip lifecycle APIs + business rule validations	Trip Dispatcher screen (Create Trip form, Live Board)
4–5	Maintenance API + status automation	Maintenance screen
5–6	Fuel/Expense APIs + cost rollup + mileage calc	Fuel & Expense screen
6–7	Reports/analytics endpoint (Pandas), CSV export	Dashboard KPIs + Analytics charts (Chart.js)
7–8	Bug fixes, seed demo data, deploy	Polish, Settings/RBAC screen, final demo run-through

Enhancement features (document reminders, mileage anomaly alerts, depot-return flag) should only be started once every row in the Must-have MoSCoW list (PRD §11) is working end-to-end — they’re additive and should never risk the core demo.